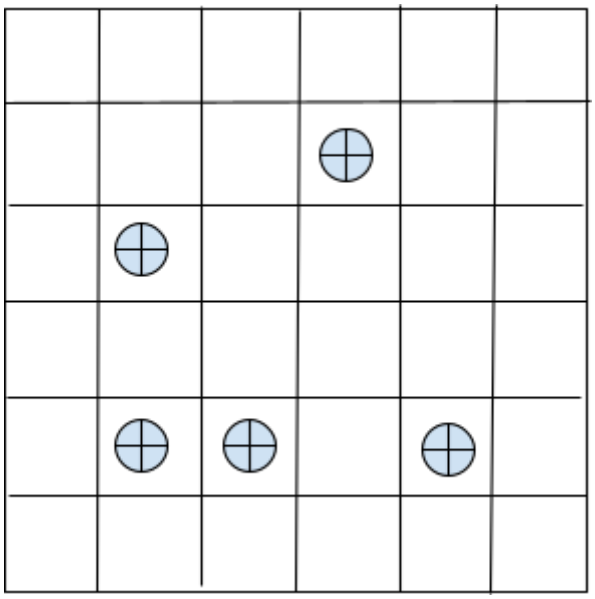
American Computer Science League

2022-2023 ● Contest 4: Targets ● Intermediate Division

PROBLEM: Given the *n* x *n* grid with target locations and with empty border cells, place arrows in every border cell and find the target(s) that get(s) hit the most. The arrow stops when it hits a target. The arrows that can be placed in any border cell are defined as A-H: **(A) ←, (B) ↑, (C) →, (D) ↓, (E) ↖, (F) ↗, (G) ↘,** and **(H)** ↙. Output the location(s) of the target(s) as 2-character strings separated by a single space in row-column order. The upper left corner is identified as location (0,0) using row-major order.

EXAMPLE:

In the grid below, the size is 6 x 6 with 5 different targets.



By checking every border position, the following arrows will hit each target:

Target	Border cell and direction	Border cell and direction	Border cell and direction	Border cell and direction	Border cell and direction	Border cell and direction	Border cell and direction	Border cell and direction
(1,3)	02G	03D	04H	10C	15A	35E	40F	53B
(2,1)	01D	03H	10G	20C	25A	30F	54E	
(4,1)	05H	30G	40C	50F	51B	52E		
(4,2)	02D	15H	20G	51F	52B	53E		
(4,4)	00G	04D	35H	45A	53F	54B	55E	

The output is "13" since that target can be hit by a maximum of 8 different arrows.

American Computer Science League

2022-2023 ● Contest 4: Targets ● Intermediate Division

INPUT: There will be one integer representing the size of the grid and another string representing the locations of the targets, each separated by a single space. We guarantee that the grid will be no larger than 10 x 10.

OUTPUT: For each grid, output the target that gets the most hits. If there is a tie, list each target in row-column order. List each target as a 2-character string, separated by a single space.

SAMPLE INPUT:

```
1. 6
   13 21 41 42 44
2. 5
   31 21 13 32 11 12
3. 10
   81 84 86 87 88 71 73 75 77 32 33 45 47 48 11 13 15 16
4. 8
   65 45 55 32 42 54 13 14 41 61 24
5. 4
   12 22 21
```

SAMPLE OUTPUT:

```
1. 13
2. 11 13
3. 16 32
4. 13 61
5. 12 21 22
```

American Computer Science League

2022-2023 ● Contest 4: Targets ● Intermediate Division

TEST DATA

TEST INPUT:

```
1. 9
   11 14 17 22 24 26 31 33 35 37 45 41 53 57 62 64 66 71 77
2. 7
   15 23 24 25 32 35 31 45 55 44
3. 6
   11 22 33 44 14 23 41
4. 10
   71 82 63 54 45 56 75 77 88 21 24 26 27 28 12 13 15 34 35 33
   37
5. 5
   11 12 13 21 22 23 31 32 33
```

TEST OUTPUT:

```
1. 45
2. 31
3. 11 14 22 33 41 44
4. 63
5. 11 13 31 33
```

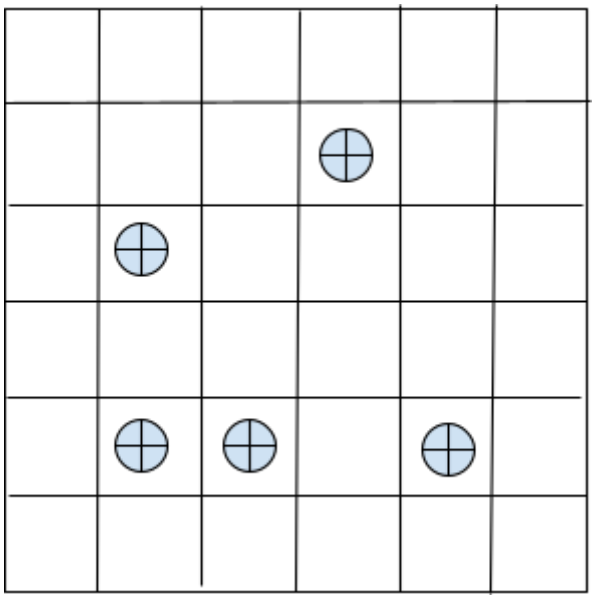
American Computer Science League

2022-2023 ● Contest 4: Targets ● Intermediate Division

问题: 给定具有目标位置和**边**界**单元格为空的** $n \times n$ **方格表**，在每个边界单元格中放置箭头，并找到被命中最多的目标。**每个箭头**击中一个目标**后就停止**。可以放置在任何边界单元格中的箭头定义为 A-H: **(A)** ←、**(B)**↑、**(C)** →、**(D)** ↓、**(E)** ↖、**(F)** ↗、**(G)** ↘ 和 **(H)** ↙。**按照行-列顺序，**将目标位置输出为用一个空格分隔**的** 2 个字符的字符串。**按照行优先的方式，左上角被识别为**位置（0, 0）。

示例:

下列 6 × 6方格表中有 5 个**不同的**目标。



通过检**验**每个**边**界位置，以下箭头击中每个目标的情况如下所示：

目标位置	边界**单元格与方向**	边界**单元格与方向**	边界**单元格与方向**	边界**单元格与方向**	边界**单元格与方向**	边界**单元格与方向**	边界**单元格与方向**	边界**单元格与方向**
(1,3)	02G	03D	04H	10C	15A	35E	40F	53B
(2,1)	01D	03H	10G	20C	25A	30F	54E	
(4,1)	05H	30G	40C	50F	51B	52E		
(4,2)	02D	15H	20G	51F	52B	53E		
(4,4)	00G	04D	35H	45A	53F	54B	55E	

输出为 “13”，因为该目标可以被**最多** 8 个不同的箭头击中。

American Computer Science League

2022-2023 ● Contest 4: Targets ● Intermediate Division

输入: 将有一个整数表示方格表的大小，另一个字符串表示目标的位置，每个位置由一个空格分隔。我们保证方格表不大于10×10。

输出: 对于每个**方格表**，输出**被**命中次数最多的目标。如果出现平局，请按行-列的顺序列出每个目标。将每个目标列为包含用一个空格分隔的 2 个字符的字符串。

样本输入:

```
1. 6
   13 21 41 42 44
2. 5
   31 21 13 32 11 12
3. 10
   81 84 86 87 88 71 73 75 77 32 33 45 47 48 11 13 15 16
4. 8
   65 45 55 32 42 54 13 14 41 61 24
5. 4
   12 22 21
```

样本输出:

```
1. 13
2. 11 13
3. 16 32
4. 13 61
5. 12 21 22
```

American Computer Science League

2022-2023 ● Contest 4: Targets ● Intermediate Division

测试数据

测试输入:

```
1. 9
   11 14 17 22 24 26 31 33 35 37 45 41 53 57 62 64 66 71 77
2. 7
   15 23 24 25 32 35 31 45 55 44
3. 6
   11 22 33 44 14 23 41
4. 10
   71 82 63 54 45 56 75 77 88 21 24 26 27 28 12 13 15 34 35 33
   37
5. 5
   11 12 13 21 22 23 31 32 33
```

测试输出:

```
1. 45
2. 31
3. 11 14 22 33 41 44
4. 63
5. 11 13 31 33
```